

STRICTLY CONFIDENTIAL — DO NOT DISTRIBUTE

Property of Thrithin Chand

This document and all concepts described herein are confidential and proprietary.

Unauthorised reproduction, distribution, or disclosure is prohibited.

SAROS

Secure Automotive Real-Time Operating System

An architectural overview

Thrithin Chand

Automotive Cybersecurity Engineer

Version 1.1 | May 2026

CONFIDENTIALITY NOTICE — *proprietary technical and commercial information that is the sole property of the author. It is provided for review purposes only. Recipients must not reproduce, forward, photograph, screenshot, paraphrase in writing, or otherwise transmit any portion of this document or its contents to any third party. Recipients must not implement, derive, or develop any product or technology based on the architecture described herein. Receipt of this document does not constitute any licence, transfer, or grant of intellectual property rights. The author reserves all rights, including the right to seek legal remedies for any unauthorised use or disclosure.*

STRICTLY CONFIDENTIAL — DO NOT DISTRIBUTE

1. Introduction

SAROS — Secure Automotive Real-Time Operating System — is an architectural concept for an embedded operating system in which security is treated as a first-class kernel concern rather than a middleware add-on. Every inter-task message is cryptographically authenticated by the kernel. Every binary is verified before execution. Every debug session is cryptographically authorised. Every firmware update is signed and version-monotonic. Every security event is recorded in a tamper-evident audit log.

Underpinning all of this is a Crypto Abstraction Layer (CAL) that gives the same source code portability across NXP CAAM, Renesas SCE, STMicroelectronics STSAFE, Espressif ESP32 hardware AES, and a software fallback using mbedTLS. The same architecture is designed to be algorithm-agile — when post-quantum cryptography becomes operationally necessary, only the CAL backend changes, not the security services that sit above it.

This document presents the SAROS architecture. It is intended as a technical reference for readers who want to understand how SAROS is designed and what it is intended to achieve.

2. The Problem

A modern vehicle contains between 50 and 150 ECUs communicating over CAN, CAN-FD, and Automotive Ethernet. Each ECU runs firmware that can be updated, debugged, and configured throughout the vehicle lifecycle — creating a substantial attack surface that regulatory frameworks now require OEMs to address systematically.

UNECE WP.29 Regulations R155 and R156 are in force for new vehicle type approvals in most major markets, requiring cybersecurity management systems and software update management systems respectively. ISO/SAE 21434 has become the de facto cybersecurity engineering standard for road vehicles, mandating evidence of cybersecurity rigour across the full ECU lifecycle: boot, runtime communication, debug access, firmware update, and incident response.

The existing OS and middleware landscape addresses parts of this requirement but rarely the whole. Most security mechanisms are implemented as middleware above a general-purpose RTOS, or as supplier-specific custom code on bare metal. The result is inconsistent security posture across ECU fleets, duplicated engineering effort, and an audit and certification burden that grows with every new programme.

2.1 Where Existing Solutions Fall Short

Solution Category	Coverage Gap
General-purpose RTOS (FreeRTOS, Zephyr)	No kernel-native message authentication; partial or absent secure boot; no integrated debug authorisation; security must be assembled by integrator
AUTOSAR Classic + SecOC module	Authentication is middleware above the OS, not kernel-enforced; tightly coupled to specific hardware; closed-source and configuration-heavy
Safety-certified RTOS (QNX, VxWorks, PikeOS, INTEGRITY)	Strong security and certification stories, but cryptographic functions are bound to specific HSMs; cross-platform porting is expensive; commercial licensing model
Bare-metal custom firmware	All security mechanisms re-implemented per programme; no uniform policy; high defect risk; certification evidence assembled from scratch each time

3. SAROS Architecture

SAROS is structured as a layered microkernel. Each layer accesses only the layer immediately beneath it. The Hardware Abstraction Layer is the only layer permitted to read or write hardware registers; the Crypto Abstraction Layer is the only path through which cryptographic operations are performed. Application code accesses operating system functionality exclusively through kernel-provided APIs.

3.1 Layer Stack

Layer	Components	Privilege
7 — Application	Vehicle logic, OTA agent, diagnostic client, supplier apps	User space
6 — System Services	OTA service, UDS (ISO 14229), Key Management, Audit Log, VSOC agent, Crypto Service (app-facing)	Service privilege
5 — Security Services	Secure Boot, SecOC MAC Engine, Secure Debug, Secure Flashing	Kernel privilege
4 — Kernel Core	Preemptive scheduler, Authenticated IPC, MPU manager, Task manager, Kernel IDS	Kernel privilege
3 — Crypto Abstraction Layer	Uniform API; backends: CAAM, SCE, STSAFE, ESP32 HW AES, mbedTLS	Kernel privilege ONLY
2 — Hardware Abstraction Layer	Timer, UART, CAN-FD, Flash, OTP fuses, interrupt controller	Kernel privilege

1 — Hardware	SoC, OTP fuses, flash, CAN bus, JTAG, hardware crypto engine	—
-----------------	--	---

3.2 Architectural Principles

- Security is enforced at the kernel layer. Application or middleware code cannot bypass it.
- Hardware access is mediated. Only the HAL touches registers; only the CAL touches crypto engines.
- The CAL is kernel-only. Applications never invoke CAL functions directly. All cryptographic operations requested by applications are mediated through the Crypto Service (see Section 6).
- Portability through abstraction. The CAL allows the same security services to run on different SoCs without modification.
- Algorithm-agility. Cryptographic primitives can be replaced — including by post-quantum algorithms — by changing only the CAL backend.
- Trust hierarchy. Every task carries a trust level determined at load time by signature verification; the kernel enforces communication policy between levels.

4. The Five-Phase Security Lifecycle

SAROS covers every phase of an ECU's security lifecycle. The phases are designed to be contiguous — there is no point at which the ECU is unprotected, and no phase relies on assumptions established outside the kernel.

Phase	Mechanism	When Active
Power-on	Secure Boot — verified chain of trust	Every boot
Runtime	SecOC MAC — every IPC message authenticated	Continuously
Access	Secure Debug — challenge-response authorisation	On demand, time-limited
Update	Secure Flashing — signed, verified firmware only	OTA or workshop
Evidence	Tamper-Evident Audit Log — CMAC-signed records	Continuously

4.1 Secure Boot

Secure Boot establishes a verified chain of trust from the moment power is applied. The boot ROM (immutable, factory-burned) verifies the Stage 1 Bootloader. Stage 1 verifies Stage 2. Stage 2 verifies the SAROS kernel image. The kernel verifies each user-space application before loading it. Any verification

failure halts the system — there is no partial boot, no silent fallback, no recovery path other than re-flashing via the Secure Flashing mechanism.

Cryptographic primitives

- ECDSA P-256 signature verification using OEM public keys with digests stored in OTP fuses.
- SHA-256 hash check of each image before execution.
- Anti-rollback enforcement — firmware version stored in OTP fuse can only increase, never decrease.
- All cryptographic verification performed via the HSM through the CAL — private keys never exposed to the CPU.

4.2 SecOC MAC (Secure Onboard Communication)

SecOC is integrated directly into the SAROS IPC layer. Every message exchanged between tasks — whether on the same ECU or across ECUs — carries a cryptographic Message Authentication Code computed using AES-128-CMAC. The receiving kernel verifies the MAC before delivering the message to the destination task. A task attempting to send a raw, unauthenticated message cannot do so — the kernel will not let it.

Protocol Data Unit structure

Field	Size	Purpose
Payload	Up to 64 bytes	Message content
Signal ID	1 byte	Identifies the signal type
Freshness Counter	4 bytes	Anti-replay protection
AES-128-CMAC (truncated)	4, 8, or 16 bytes (configurable)	Authentication tag

Key properties

- AES-128-CMAC computed via the CAL — hardware-accelerated on supported platforms.
- Freshness counter persisted in NVM — survives power cycles, prevents replay across reboots.
- Configurable MAC truncation — 4 bytes for bandwidth-constrained signals, up to 16 for full security.
- Constant-time MAC comparison — prevents timing side-channel attacks.
- MAC covers payload, signal ID, and freshness counter — no field can be tampered without detection.

4.3 Secure Debug

All debug interfaces — JTAG, UART debug shell, UDS diagnostic commands — are locked by default in production builds. Access requires cryptographic challenge-response authentication using the engineer's private key. Sessions are time-limited and automatically close on timeout or power cycle. Repeated failed attempts trigger a cooldown lockout.

Authentication flow

1. Engineer tool sends a debug unlock request.
2. ECU generates a 32-byte random challenge via hardware RNG.
3. Engineer signs the challenge with their ECDSA private key.
4. ECU verifies the signature against the registered public key in the HSM.
5. On success — a session token is issued, debug access granted for the configured duration (default 5 minutes).
6. On failure — the attempt is logged; repeated failures trigger an extended cooldown.

Granular access levels

Access Level	Who	Permissions
Level 1 — Read Only	Supplier engineer	Read logs, read ECU state
Level 2 — Diagnostic	OEM engineer	UDS diagnostics, read memory
Level 3 — Full Debug	OEM security team	Full JTAG, memory write, flash
Level 4 — Factory	Production line	Key provisioning, fuse programming

At end-of-life, debug can be permanently disabled by blowing a designated OTP fuse — after which no debug authorisation is possible regardless of credentials.

4.4 Secure Flashing

Only authenticated, signed, and verified firmware can be written to an ECU running SAROS — whether via physical OBD at a workshop or remotely via OTA. Both paths use the same Secure Flash Service with identical cryptographic checks.

Secure Flash Package (.sfp) format

Section	Contents	Purpose
Header	Magic bytes, format version, target ECU ID, HW ID, firmware version, min version, timestamp	Identification and anti-rollback

Crypto	SHA-256 hash, ECDSA P-256 signature, signer ID	Integrity and authenticity
Body	Firmware binary (optionally AES-256 encrypted)	Payload
Footer	AES-CMAC over entire package	End-to-end integrity

Flashing flow

7. Authentication — challenge-response with OEM/engineer key before any data transfer.
8. Package verification — magic, target ECU ID, hardware ID, ECDSA signature, SHA-256 hash.
9. Anti-rollback check — new version must be greater than or equal to OTP fuse version.
10. Write to inactive partition — erase partition B, write chunk by chunk with running hash.
11. Post-write verification — recompute hash, compare against package hash.
12. Commit and reboot — update OTP version, signal bootloader, secure boot verifies new image on next boot.

A/B partition strategy

- Partition A — currently running firmware (untouched during update).
- Partition B — receives new firmware during update.
- On successful secure boot of B — B becomes active, A becomes backup.
- On any failure — ECU automatically boots from A, no manual intervention needed.
- Zero brick risk — a failed update always recovers to the last known good firmware.

4.5 Tamper-Evident Audit Log

Every security-relevant event is recorded in a dedicated audit log partition. Each entry is signed with AES-CMAC and includes a monotonically increasing freshness counter — making any deletion, reordering, or substitution of entries detectable on integrity verification.

Logged events

- Every boot — success or failure with reason.
- Every MAC verification failure on inter-task or inter-ECU messages.
- Every debug authorisation attempt — success or failure.
- Every flash operation — start, completion, abort.
- Every IDS detection event.
- Every cryptographic key load or erase operation.
- Every application-initiated crypto operation via the Crypto Service.

Properties

- Each entry: timestamp + event type + details + sequence counter + CMAC signature.
- Log partition cannot be erased without authentication at the highest access level.
- Log integrity is verifiable by the OEM at any time — supports ISO 21434 audit requirements.
- Entries are exportable via a secure UDS service to a VSOC platform for fleet-wide monitoring.

5. Crypto Abstraction Layer

The CAL is the portability backbone of SAROS. It provides a single uniform crypto API that Secure Boot, SecOC, Secure Debug, and Secure Flashing all call — none of them touch hardware crypto engines directly. The CAL routes each operation to the correct backend at runtime based on what hardware is present.

Critically, the CAL is a kernel-internal facility. Applications cannot invoke CAL functions directly — see Section 6 for how applications request cryptographic operations.

5.1 Supported Backends

Platform	Crypto Engine	Key Protection	RNG	Production Grade
NXP i.MX 8 QM	CAAM	Hardware HSM	True HW RNG	Yes
Renesas RH850	SCE / HSM	Wrapped keys	True HW RNG	Yes
STM32 + STSAFE	Secure element	Secure element	HW RNG	Yes
Espressif ESP32	HW AES accelerator	RAM only	HW RNG	Partial
Generic ARM / AVR	mbedTLS (software)	RAM — exposed	SW entropy	No

5.2 Uniform CAL API

All kernel-resident security components call only these functions. The backend implementation is completely transparent to the caller.

- `crypto_aes_cmac(key_id, data, len, mac_out)` — AES-128-CMAC computation
- `crypto_aes_encrypt(key_id, plain, len, cipher_out)` — AES encryption
- `crypto_sha256(data, len, hash_out)` — SHA-256 hash
- `crypto_rng(buf, len)` — cryptographically secure random bytes
- `crypto_ecdsa_sign(key_id, hash, sig_out)` — ECDSA P-256 signature

- `crypto_ecdsa_verify(key_id, hash, sig)` — ECDSA P-256 verification
- `crypto_key_load(key_id, key_data, len)` — load key into engine
- `crypto_key_delete(key_id)` — securely erase key from engine

5.3 Backend Selection

Method	Mechanism	Use Case
Compile time	<code>saros.config</code> + Makefile flags	Production build for known hardware
Boot time probe	<code>cal_init()</code> probes hardware registers	Development, multi-platform builds
Runtime switch	<code>crypto switch</code> command in debug shell	Development and testing only

6. How Applications Access Cryptography

A common question about the SAROS architecture is: how do applications use cryptography if they cannot call the CAL directly? This section answers that question with a concrete worked example.

6.1 The Design Decision — CAL Is Not Application-Accessible

In SAROS, application code never invokes CAL functions directly. The CAL is a kernel-internal facility, protected by the same privilege boundary that protects the MPU manager, the scheduler, and the hardware abstraction layer. There are three reasons for this design decision:

13. Smallest attack surface — if applications cannot call crypto primitives directly, they cannot misuse them. They cannot pick weak modes, reuse nonces, leak keys through timing side channels, or accidentally bypass freshness counters.
14. Key isolation — application code never holds raw key material. Keys live inside the HSM or the kernel's protected memory. An application requesting a signature gets the signature, never the key.
15. Cleaner certification scope — under ISO 21434 and ISO 26262, the cryptographic boundary is sharply defined. Only the kernel and its security services are in the cryptographic trust boundary; applications are outside it.

6.2 The Application-Facing Path — The Crypto Service

Applications that legitimately need cryptographic operations — for example, an OTA agent verifying a server certificate, a diagnostic client encrypting a payload to a backend, or a VSOC agent signing a

telemetry report — request those operations through the Crypto Service, a Layer 6 system service that exposes a constrained subset of cryptographic operations as kernel syscalls.

The Crypto Service syscall interface

- `crypto_svc_sign(handle, data, len, sig_out)` — request a signature using a key the application is permitted to use
- `crypto_svc_verify(pubkey_handle, data, len, sig)` — verify a signature against a known public key
- `crypto_svc_encrypt_aead(handle, plain, len, aad, cipher_out)` — authenticated encryption (AES-GCM or similar)
- `crypto_svc_decrypt_aead(handle, cipher, len, aad, plain_out)` — authenticated decryption
- `crypto_svc_hash(data, len, hash_out)` — compute a hash
- `crypto_svc_random(buf, len)` — request cryptographically secure random bytes

What the Crypto Service enforces on every call

- Key access control — each key has an Access Control List. The Crypto Service checks that the calling application's identity (signed binary identity, trust level) is permitted to use the requested key. If not, the call fails.
- Safe defaults — only authenticated encryption modes are exposed (AES-GCM, AES-CCM). Raw AES-ECB or AES-CBC is not available. Only SHA-256 or stronger. No deprecated algorithms.
- Key material never leaves the kernel — keys are referenced by handle, never by value. The application cannot extract a key under any circumstances.
- Audit logging — every Crypto Service call is recorded in the tamper-evident audit log with the caller identity, operation type, key handle, and outcome.
- Rate limiting — repeated failed authentication attempts trigger lockout to prevent brute-force attacks on key access.

6.3 Worked Example — An OTA Agent Verifying a Server Certificate

Consider an over-the-air update agent running as a Layer 7 application. It needs to verify that an OTA package downloaded from an OEM server was signed by a trusted certificate authority. Here is the complete flow:

Step 1 — Application makes a syscall

The OTA agent has the server's signature and the data that was signed. It invokes the Crypto Service via a syscall:

```
// Application code (Layer 7) — OTA agent
```

```
result = crypto_svc_verify(
    OEM_ROOT_CA_PUBKEY,    // Public key handle (not the key itself)
    ota_package_data,      // Data that was signed
    ota_package_len,       // Length of data
    server_signature);     // Signature to verify
```

Step 2 — Kernel receives the syscall and validates the request

The kernel transitions to privileged mode and the Crypto Service handler runs. It performs the following checks before any crypto operation is invoked:

16. Caller identity check — what application is calling? Its trust level is read from the task control block.
17. Key ACL check — is the OEM_ROOT_CA_PUBKEY handle accessible to this caller? The Key Management Service holds an ACL: "OEM_ROOT_CA_PUBKEY is readable by tasks with trust level ≥ 3 (OEM Apps) and the OTA agent specifically." Check passes.
18. Operation policy check — is verification the right operation for a public key? Yes. (Signing would have been rejected — public keys cannot sign.)
19. Rate limit check — has this caller exceeded its allowed crypto operations per second? No.
20. Audit entry pre-creation — a pending log entry is opened.

Step 3 — Kernel invokes the CAL (the only place CAL is called)

Once all validation passes, the Crypto Service makes the actual CAL call:

```
// Kernel code (inside Crypto Service) — invokes CAL
ret = crypto_ecdsa_verify(
    OEM_ROOT_CA_PUBKEY,    // Resolved to actual key in HSM
    sha256_of_data,        // CAL handles hashing internally
    server_signature);
```

Step 4 — CAL routes to the platform backend

The CAL determines which backend is active and routes the call. On an NXP i.MX 8 ECU, the call goes to the CAAM backend; on an ESP32 development board, it goes to the mbedTLS software backend. The application has no visibility into this. The application called `crypto_svc_verify`; the kernel called `crypto_ecdsa_verify`; the CAL invoked the appropriate hardware engine. The result returns through the same chain in reverse.

Step 5 — Result returned to application

The Crypto Service finalises the audit log entry (success or failure, with the verified signature's metadata but never the key), and returns the boolean result to the application. The application now knows

whether the OTA package is authentic. It never touched a key, never called the CAL, never had visibility into which hardware did the work.

6.4 Why This Matters

This architecture is what makes SAROS auditable. When a vehicle security analyst asks "who can use this key, when was it used, what for, and was it ever used incorrectly" — the audit log answers all four questions because every cryptographic operation passes through the Crypto Service. When a malicious application is loaded onto the ECU and tries to extract a key, it cannot — there is no syscall that returns key material. When a future application is added that needs cryptography, it integrates by requesting a key handle and an ACL entry, not by linking against a crypto library.

In SAROS, an application that wants cryptography does not get a crypto library. It gets a contract with the kernel. The kernel performs the cryptography on the application's behalf, under enforced policy, with complete audit. This is the difference between security as a feature and security as an architectural property.

7. Post-Quantum Cryptographic Migration

Vehicles built today will be on the road in 2040 and beyond. Cryptographic algorithms in widespread use today — RSA, ECDSA, ECDH, AES, SHA-2 — face known threats from sufficiently large quantum computers. While quantum computers capable of breaking ECDSA P-256 do not exist today, the cryptographic research community and standards bodies have moved decisively to standardise post-quantum alternatives over the past five years.

An ECU designed in 2026 must therefore be capable of being updated to use post-quantum algorithms during its operational lifetime, without redesigning the operating system, the security services, or the application software.

SAROS achieves this through the Crypto Abstraction Layer.

7.1 What Algorithms Are Available Today

The National Institute of Standards and Technology (NIST) finalised the first set of post-quantum standards in August 2024:

Standard	Algorithm	Purpose	Status (2026)
FIPS 203 (ML-KEM)	Module-Lattice-Based Key Encapsulation	Key exchange / encapsulation (replaces ECDH, RSA-OAEP)	Standardised, production-ready
FIPS 204 (ML-DSA)	Module-Lattice-Based Digital	Digital signatures (replaces	Standardised,

	Signature Algorithm	ECDSA, RSA-PSS)	production-ready
FIPS 205 (SLH-DSA)	Stateless Hash-Based Digital Signature Algorithm	Backup signature scheme (hash-based, conservative)	Standardised, production-ready
NIST round 4 (HQC, BIKE, others)	Code-based KEM alternatives	Algorithm diversity for KEM	Under evaluation

The classical algorithms used in SAROS today — ECDSA P-256 for signatures, AES-128-CMAC for message authentication, SHA-256 for hashing — are secure against current threats and remain the operational choice for ECUs entering service in 2026. The post-quantum standards are available and ready, but not yet mandatory in the automotive industry.

7.2 The Migration Problem

Consider a vehicle ECU programme launched in 2026 using ECDSA P-256 for secure boot signature verification and for OTA package signing. In 2032, a regulatory or risk-driven decision is made to migrate the entire fleet to ML-DSA-65 signatures. Without an abstraction layer, this migration requires:

- Modifying the secure boot implementation to call ML-DSA verification instead of ECDSA verification.
- Modifying the OTA update verification code in the same way.
- Modifying every other location in the operating system or middleware where signature verification is called.
- Re-testing, re-validating, and re-certifying every modified component.
- Re-issuing every signing key in the OEM PKI.

With SAROS, the migration is dramatically smaller. The OS, the security services, and the applications all consume cryptography through stable APIs — `crypto_ecdsa_verify` in the CAL, `crypto_svc_verify` in the Crypto Service. The implementation behind those APIs is what changes.

7.3 Worked Example — Migrating Secure Boot from ECDSA P-256 to ML-DSA-65

Here is exactly what happens, step by step, when an OEM decides to migrate the SAROS secure boot mechanism from classical ECDSA P-256 to post-quantum ML-DSA-65.

Step 1 — Add ML-DSA support to the CAL

A new function is added to the CAL API:

```
// New CAL function added
```

```
crypto_pq_signature_verify(
    PQ_ALGORITHM_ML_DSA_65,    // Algorithm selector
    key_id,                    // Public key handle
    data, len,                  // Data that was signed
    signature, sig_len);        // Signature to verify
```

Each platform backend implements this new function. On NXP CAAM, the implementation may use a CAAM firmware update that adds ML-DSA support. On platforms without hardware ML-DSA, the mbedTLS or liboqs software backend handles it. On STSAFE, an updated secure element firmware provides it. The same API; different backends; the security service does not care.

Step 2 — Add a unified verification function in the CAL

A higher-level CAL helper is added that automatically selects between classical and post-quantum verification based on the key type:

```
crypto_verify_signature(
    key_id,                    // Key tells us the algorithm
    data, len,
    signature, sig_len);
```

Internally, this function reads the algorithm tag stored with the key and dispatches to either `crypto_ecdsa_verify` or `crypto_pq_signature_verify`. The secure boot service, the secure flashing service, and any other consumer now calls only this one function.

Step 3 — Issue new keys in the OEM PKI

The OEM generates new ML-DSA-65 signing keys and provisions the public key digests into a new set of OTP fuses (or into a fuse slot reserved during initial manufacture). Critically, this does not invalidate the existing ECDSA keys — both algorithms can coexist during the transition period.

Step 4 — Sign new firmware packages with ML-DSA

The OEM build pipeline is updated. New firmware packages are signed with the new ML-DSA-65 keys instead of ECDSA. The .sfp package header now contains a signer ID that points to the ML-DSA public key in the fuse.

Step 5 — Deploy the CAL update via OTA

The updated CAL — containing the new ML-DSA backend implementation — is delivered to the fleet through the existing Secure Flashing mechanism. This update is itself still signed with the existing ECDSA key, since the existing CAL on the ECU only knows ECDSA. Once the update is installed and verified, the ECU now has both ECDSA and ML-DSA capability.

Step 6 — Subsequent updates use ML-DSA

The next firmware package after the CAL update is signed with ML-DSA. The secure boot service calls `crypto_verify_signature`; the CAL sees the ML-DSA key handle and routes to the ML-DSA backend; verification succeeds; the new firmware boots. The OEM has migrated the fleet to post-quantum signatures.

Step 7 — Deprecate ECDSA when ready

After a transition period during which both algorithms are accepted, the OEM signs a final firmware package that removes ECDSA verification capability from the CAL. From that point onward, only ML-DSA-signed firmware is accepted. The migration is complete.

7.4 What Did NOT Have to Change

This is the part that matters. During the entire migration above:

- The Secure Boot service implementation did not change — it still calls `crypto_verify_signature`.
- The Secure Flashing service did not change.
- The .sfp package format did not change — only the signature algorithm referenced by the signer ID.
- The kernel IPC layer, SecOC engine, and audit log did not change.
- No application code changed. Applications using `crypto_svc_verify` continue to work — the Crypto Service routes through the updated CAL.
- ISO 21434 re-certification scope is bounded to the CAL backend and the OEM key provisioning process — not the entire operating system.

In contrast, a system without this abstraction would require modification, re-testing, and re-certification of every component that performs signature verification — potentially the entire firmware stack.

7.5 Future Algorithms — The Same Pattern Applies

ML-DSA and ML-KEM are the current post-quantum standards. The cryptographic landscape will continue to evolve. New algorithms will be standardised. Some currently approved algorithms may be deprecated if cryptanalysis discovers weaknesses. Hybrid schemes — combining classical and post-quantum algorithms for defence in depth — are already being deployed in some sectors and may become standard practice.

Each of these transitions, in SAROS, follows the same pattern as the ECDSA-to-ML-DSA example above:

21. Add support for the new algorithm in the CAL.
22. Issue new keys in the OEM PKI.
23. Deliver the CAL update via Secure Flashing.

24. Sign subsequent firmware with the new algorithm.

25. Deprecate old algorithms when policy allows.

The architecture above the CAL is unaffected. This is what "algorithm-agility" means in SAROS, and it is the principal reason the architecture is designed to outlive any specific cryptographic primitive.

8. Kernel Core Features

8.1 Authenticated IPC

This is the most distinctive feature of the SAROS kernel. In every other RTOS, inter-process messages are unauthenticated by default. In SAROS, every IPC message automatically carries SecOC authentication — the kernel appends the freshness counter, computes the CMAC via the CAL, and only delivers messages whose MAC and freshness counter both verify. Authentication is transparent to the application; the application does not call it explicitly and cannot opt out.

- Send path: kernel appends freshness counter, computes CMAC, transmits signed PDU.
- Receive path: kernel verifies CMAC and freshness before delivery.
- Spoofed or replayed messages are dropped and logged — they never reach the application.
- Authentication is enforced by the kernel, not by the application — implementation defects in application code cannot create authentication bypasses.

8.2 Trust Level System

Every task in SAROS operates at a defined trust level. The kernel enforces communication boundaries between levels — a lower-trust process cannot impersonate a higher-trust one, because the cryptographic authentication of section 8.1 makes impersonation cryptographically detectable.

Trust Level	Who	Access Rights
Level 0 — ROM	Immutable boot code	Absolute, unrestricted
Level 1 — Kernel	SAROS kernel (verified by ROM)	Full hardware access, full CAL access
Level 2 — Trusted Services	OTA, Key Mgmt, Audit, Crypto Service	Kernel APIs, CAL access (mediated), flash
Level 3 — OEM Apps	Signed by OEM key	Crypto Service syscalls, IPC, diagnostics
Level 4 — Supplier Apps	Signed by supplier key	Limited IPC, limited Crypto Service access
Level 5 — Sandboxed	Untrusted or test code	Isolated, no system access, no crypto

8.3 Kernel Intrusion Detection

The SAROS kernel monitors its own runtime behaviour for anomalies indicative of attack or compromise. Detected events are logged to the audit log and can be escalated to a gateway ECU or VSOC platform where network connectivity is available.

- Unexpected message patterns on the CAN bus — possible injection attack.
- Task accessing memory outside its MPU region — possible exploit attempt.
- Multiple consecutive MAC failures on the same signal — possible spoofing attack.
- Debug unlock attempts outside the maintenance window — possible physical attack.
- Non-monotonic freshness counter transitions — possible replay attack.
- Bootloader access outside the scheduled update window — possible manipulation.
- Repeated Crypto Service authentication failures from the same application — possible key-guessing attack.

9. Where SAROS Sits

The automotive secure-OS landscape today has high-end commercial solutions used in production by major OEMs, and low-end open-source RTOSes without integrated security. SAROS occupies the gap between them — a unified, kernel-native, portable, crypto-agile architecture intended to bring lifecycle security to a wider range of ECU programmes.

Solution	Position
Vector MICROSAR, ETAS RTA-OS, EB Tresos	Industry-standard AUTOSAR Classic stacks with SecOC modules. Closed source, high licensing cost, hardware-coupled.
QNX Safety, VxWorks Cert, PikeOS, INTEGRITY	Certified safety/security OSes with strong production track records. Premium pricing; cryptographic functions bound to specific HSMS.
Zephyr RTOS	Open source, growing community. Lacks first-class SecOC, no unified crypto abstraction layer, only partial secure boot.
FreeRTOS	Ubiquitous, simple, free. No native security primitives at all.
SAROS (this work)	Open-core architecture with kernel-enforced SecOC, portable Crypto Abstraction Layer, kernel-only CAL with mediated application access, post-quantum-ready by design, integrated five-phase security lifecycle.

10. Development Roadmap

SAROS is currently at the architectural design and prototype-planning stage. The roadmap below describes the phased build-out from foundation to first release.

Phase	Focus	Deliverable
Phase 1 — Foundation	Bare metal boot, UART, basic scheduler	Kernel skeleton on first target platform
Phase 2 — CAL	Crypto Abstraction Layer with mbedTLS backend	AES-CMAC, SHA-256, ECDSA working through the abstraction
Phase 3 — Secure Boot	ECDSA verification, image header format, anti-rollback fuse	Verified chain of trust from ROM to user space
Phase 4 — SecOC IPC	Authenticated IPC, freshness manager, NVM persistence	Kernel-enforced message authentication
Phase 5 — Crypto Service	App-facing syscall API, key ACL, audit integration	Applications can request crypto under enforced policy
Phase 6 — Secure Debug	Challenge-response auth, session management, access levels	Locked-by-default debug interfaces
Phase 7 — Secure Flash	SFP package format, A/B partition, OTA agent	End-to-end signed firmware update
Phase 8 — Hardware Ports	NXP CAAM, Renesas SCE CAL backends	Validation of CAL portability claim
Phase 9 — Audit + IDS	Tamper-evident log, kernel IDS, VSOC reporting	Forensic evidence support for ISO 21434
Phase 10 — PQ Backend	ML-DSA / ML-KEM CAL backend (software, then HW when available)	Validation of post-quantum migration story
Phase 11 — Release	Documentation, public release	Open-core release

First target platform: ESP32 with mbedTLS software crypto backend — affordable, well-documented hardware with dual-core support and WiFi/BLE for OTA testing. Allows full OS development and validation before porting to NXP i.MX 8 QM and Renesas RH850.

11. Design Principles

A few principles underlie every architectural decision in SAROS. They are stated here for clarity.

26. Security is a kernel responsibility. If an application can bypass it, it isn't security.
27. Hardware access is mediated. The HAL is the only path to registers; the CAL is the only path to crypto engines; the Crypto Service is the only path from applications to cryptography. No exceptions.

28. Abstraction enables longevity. Portability across SoCs and algorithm-agility for post-quantum migration are designed in from day one, not added later.
29. Every security event is recorded. If an incident occurs, the audit log is the source of truth.
30. Failure modes are designed. There is no silent degradation, no partial boot, no recovery to unsigned code. Verification failures halt; update failures revert.
31. The architecture is documented for review. Anything that cannot be explained and defended in writing does not belong in the design.

12. Status and Outlook

SAROS is presently at the architectural design stage, with the v1.1. The implementation roadmap targets the ESP32 platform for the first end-to-end prototype, with subsequent ports to automotive-grade SoCs validating the Crypto Abstraction Layer's portability claim.

The architecture has been deliberately designed to be reviewable, defensible, and standards-aligned — compatible with development under an ISO/SAE 21434 cybersecurity engineering process and capable of producing the evidence artefacts required by UNECE WP.29 R155 and R156.

This document will be revised as the design evolves through implementation.

REMINDER — This document and the SAROS architecture described herein are the proprietary intellectual property of Thrithin Chand. All rights are reserved. No portion of this document may be copied, photographed, transmitted, paraphrased, summarised in writing, published, presented externally, or used as the basis for any product, design, or implementation without the prior written consent of the author. Unauthorised use will be pursued through all available legal remedies.

STRICTLY CONFIDENTIAL — DO NOT DISTRIBUTE